

Mảng tiền tố 2D & Difference Array

Vấn đề

Trong chapter 2 bạn đã học prefix sum 1D – tính tổng đoạn $[l, r]$ trong $O(1)$ sau tiền xử lý $O(n)$. Đây là một trong những kỹ thuật đơn giản nhất nhưng mạnh nhất trong lập trình thi đấu. Bài này mở rộng ý tưởng đó lên 2 chiều (truy vấn tổng hình chữ nhật trong ma trận) và giới thiệu kỹ thuật đối ngẫu: difference array – cho phép cộng vào cả một đoạn trong $O(1)$.

1. Prefix Sum 2D – tổng hình chữ nhật

Đề: Cho ma trận a kích thước $n \times m$. Có q truy vấn, mỗi truy vấn cho hình chữ nhật (r_1, c_1, r_2, c_2) , hỏi tổng các phần tử trong hình chữ nhật đó.

Ý tưởng: Định nghĩa $S[r][c]$ là tổng các phần tử trong hình chữ nhật từ $(1, 1)$ tới (r, c) .

Ta tính S bằng công thức bao hàm-loại trừ:

$$S[r][c] = a[r][c] + S[r-1][c] + S[r][c-1] - S[r-1][c-1]$$

Tổng hình chữ nhật (r_1, c_1, r_2, c_2) :

$$\text{Sum} = S[r_2][c_2] - S[r_1-1][c_2] - S[r_2][c_1-1] + S[r_1-1][c_1-1]$$

Hãy hình dung công thức như bao hàm-loại trừ: bắt đầu với hình chữ nhật lớn từ $(1, 1)$ tới (r_2, c_2) (vùng xanh lá), trừ đi phần phía trên và phần bên trái (hai vùng đỏ), nhưng giao của hai phần đó – vùng xanh dương ở góc trên-trái – đã bị trừ hai lần, nên cộng lại.

Code

C++

```
void build() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            S[i][j] = a[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1];
}

int query(int r1, int c1, int r2, int c2) {
    return S[r2][c2] - S[r1-1][c2] - S[r2][c1-1] + S[r1-1][c1-1];
}
```

Python

```
# a la ma tran (n+1) x (m+1), index tu 1
S = [[0] * (m + 1) for _ in range(n + 1)]
```

```

for i in range(1, n + 1):
    for j in range(1, m + 1):
        S[i][j] = a[i][j] + S[i-1][j] + S[i][j-1] - S[i-1][j-1]

def query(r1, c1, r2, c2):
    return S[r2][c2] - S[r1-1][c2] - S[r2][c1-1] + S[r1-1][c1-1]

```

Lợi ích: Tiền xử lý $O(nm)$, mỗi truy vấn $O(1)$.

Khi nào dùng?

- Đếm số phần tử thỏa điều kiện trong vùng chữ nhật
- Tính tổng vùng chữ nhật
- Bài toán liên quan đến lưới: số ô đen trong hình chữ nhật, v.v.

2. Difference Array 1D – cộng đoạn nhanh

Đề: Cho mảng a kích thước n ban đầu toàn 0. Có q thao tác (l, r, v) : cộng v vào tất cả $a[l..r]$. Sau cùng, in mảng.

Cách ngây thơ: $O(qn)$ – chậm.

Ý tưởng difference array: Định nghĩa $d[i] = a[i] - a[i-1]$. Khi đó "cộng v vào $a[l..r]$ " tương đương:

- $d[l] += v$
- $d[r+1] -= v$

Tất cả các $d[i]$ khác không đổi! Sau khi làm hết q thao tác, lấy prefix sum của d để khôi phục a .

C++

```

long long d[N + 2] = {0};

void update(int l, int r, long long v) {
    d[l] += v;
    d[r + 1] -= v;
}

void build() {
    for (int i = 1; i <= n; i++) d[i] += d[i - 1]; // Prefix sum
    ra gia tri that
}

```

Python

```

d = [0] * (n + 2)

```

```
def update(l, r, v):
    d[l] += v
    d[r + 1] -= v

def build():
    for i in range(1, n + 1):
        d[i] += d[i - 1]
```

Tổng độ phức tạp: $O(n + q)$ – cực nhanh.

Tổng quan: Prefix sum & Difference array là đôi ngầu

- Prefix sum: nhiều truy vấn tổng đoạn, không có cập nhật.
- Difference array: nhiều cập nhật cộng đoạn, sau cùng truy vấn 1 lần.

Nếu cả hai cùng có (online) → cần Fenwick / Segment Tree (Vàng).

3. Difference Array 2D

Đề: Cộng giá trị v vào hình chữ nhật (r_1, c_1, r_2, c_2) nhiều lần. Sau cùng, in toàn ma trận.

Ý tưởng: Mở rộng difference array 1D – chỉ cần 4 cập nhật điểm.

C++

```
void update(int r1, int c1, int r2, int c2, int v) {
    d[r1][c1] += v;
    d[r2 + 1][c1] -= v;
    d[r1][c2 + 1] -= v;
    d[r2 + 1][c2 + 1] += v;
}

void build() {
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1];
}
```

Python

```
d = [[0] * (m + 2) for _ in range(n + 2)]

def update(r1, c1, r2, c2, v):
    d[r1][c1] += v
    d[r2 + 1][c1] -= v
    d[r1][c2 + 1] -= v
    d[r2 + 1][c2 + 1] += v

def build():
```

```

for i in range(1, n + 1):
    for j in range(1, m + 1):
        d[i][j] += d[i - 1][j] + d[i][j - 1] - d[i - 1][j - 1]

```

Mỗi cập nhật chỉ chạm 4 điểm $\rightarrow O(1)$ mỗi update, $O(nm)$ build.

Tại sao 4 điểm hoạt động?

Tưởng tượng "lan tỏa": dấu + lan từ (r_1, c_1) xuống dưới-phải mãi mãi. Dấu $-$ ở $(r_2 + 1, c_1)$ và $(r_1, c_2 + 1)$ chặn vùng lan ngoài hình chữ nhật. Dấu + ở góc $(r_2 + 1, c_2 + 1)$ bù lại vùng bị trừ hai lần.

4. Mẫu bài toán điển hình

Mẫu 1: Số ô đen trong hình chữ nhật

Đặt $a[i][j] = 1$ nếu ô đen, 0 nếu trắng. Tính prefix sum 2D, mỗi truy vấn $O(1)$.

Mẫu 2: Hình vuông tổng cao nhất

Tìm hình vuông kích thước k trong ma trận $n \times n$ có tổng lớn nhất. Tiền xử lý prefix sum 2D, duyệt $O((n - k)^2)$ vị trí.

Mẫu 3: Đếm số người ở mỗi điểm

Có q người, mỗi người ở vùng chữ nhật. Hỏi tại điểm (i, j) có bao nhiêu người. Difference array 2D + prefix sum.

5. Bẫy thường gặp

- Lỗi off-by-one: $r_2 + 1$ và $c_2 + 1$ phải hợp lệ. Khai báo mảng dư 2 hàng/cột để tránh out-of-bounds.
- Index 0 vs 1: Prefix sum 2D dễ nhầm. Khuyến: index từ 1, để $S[0][*] = S[*][0] = 0$.
- Tràn số (overflow): Tổng có thể rất lớn – dùng `long long`.

6. Tổng kết

KỸ THUẬT	MỤC ĐÍCH	TIỀN XỬ LÝ	MỖI THAO TÁC
Prefix sum 1D	Truy vấn tổng đoạn	$O(n)$	$O(1)$
Prefix sum 2D	Truy vấn tổng hình chữ nhật	$O(nm)$	$O(1)$
Difference 1D	Cộng đoạn (offline)	$O(n + q)$	$O(1)$ update
Difference 2D	Cộng hình chữ nhật (offline)	$O(nm + q)$	$O(1)$ update

Chỉ cần nhớ: prefix sum cho truy vấn, difference cho cập nhật, công thức bao hàm-loại trừ là chìa khóa!